# Validate Bst

AlgoExpert · Medium · Binary Search Trees

# The problem

- Every node must be strictly greater than all left-subtree values and <= all right-subtree values.

# Intuition

- Every node must lie within bounds inherited from its ancestors, not just compared with its parent.

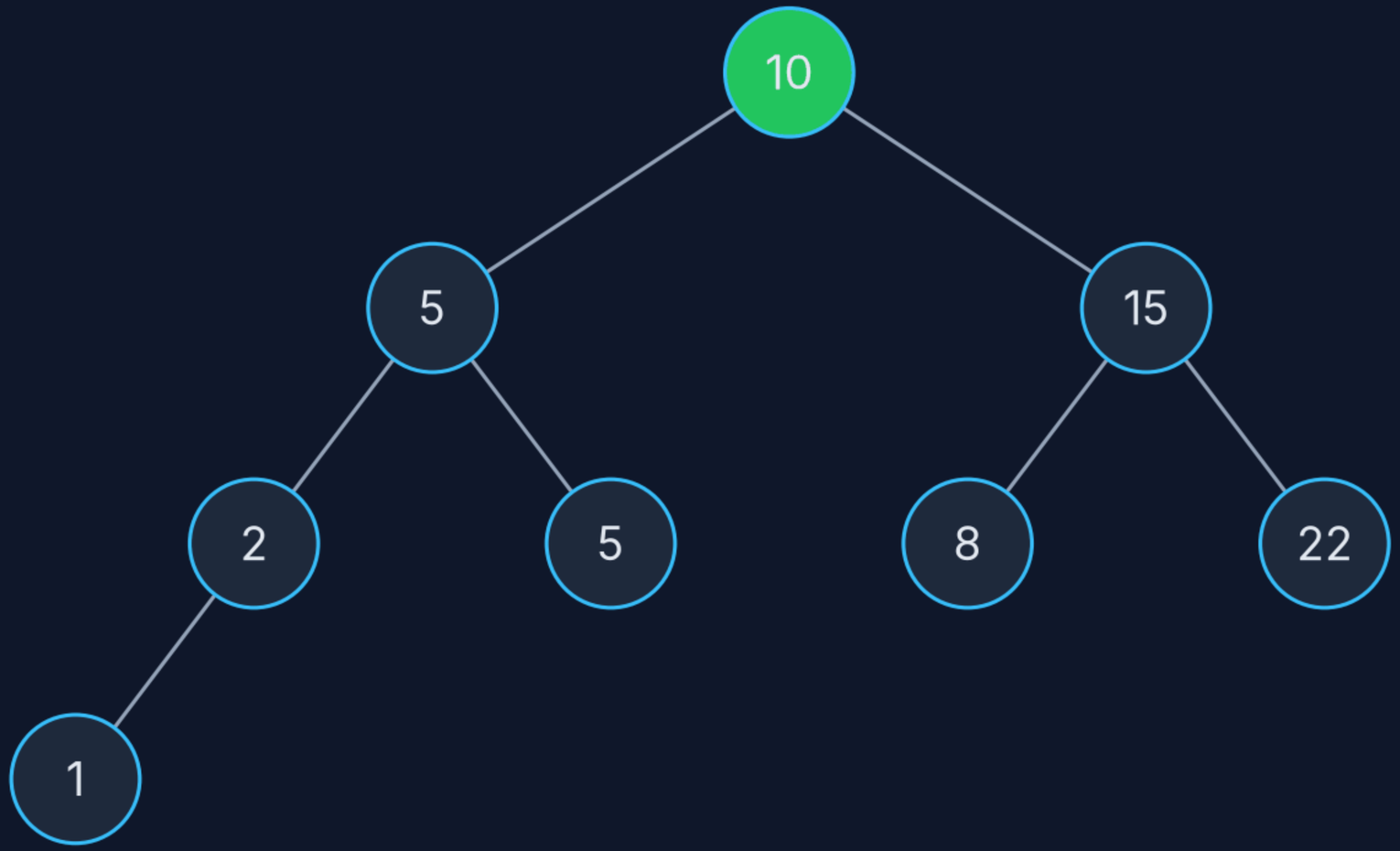
# Brute force vs optimal

- Naive: compare each node only with its children — WRONG
- In-order traversal must be sorted → O(n)
- Bounds passed down → O(n), early exit

# Approach

- 1. Start with minus and plus infinity
- 2. Check the node is within bounds
- 3. Left child tightens the upper bound
- 4. Right child tightens the lower bound

# Worked example

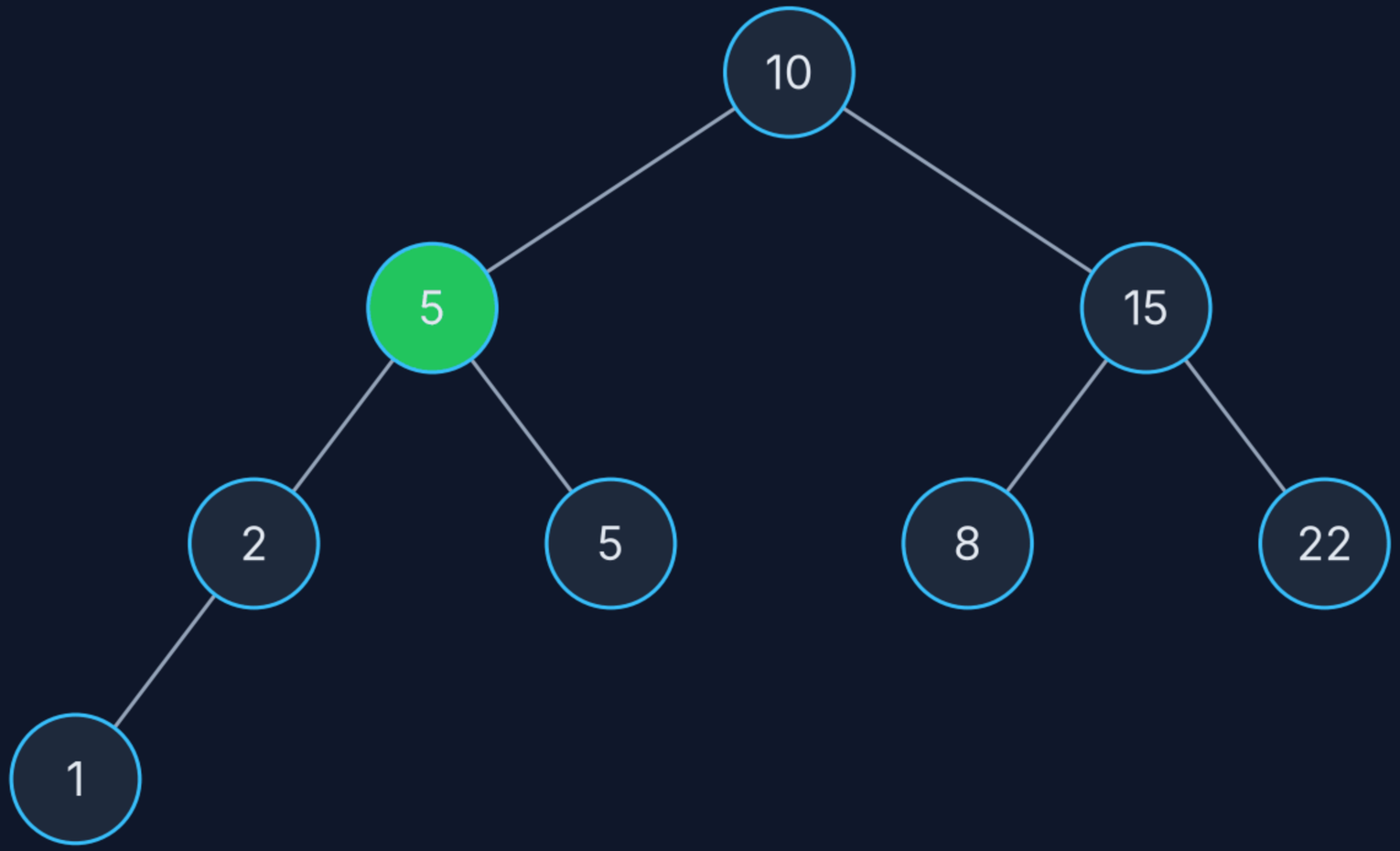



node 10

bounds [-∞, ∞)

10 must be in [-∞, ∞) → ok

# Worked example

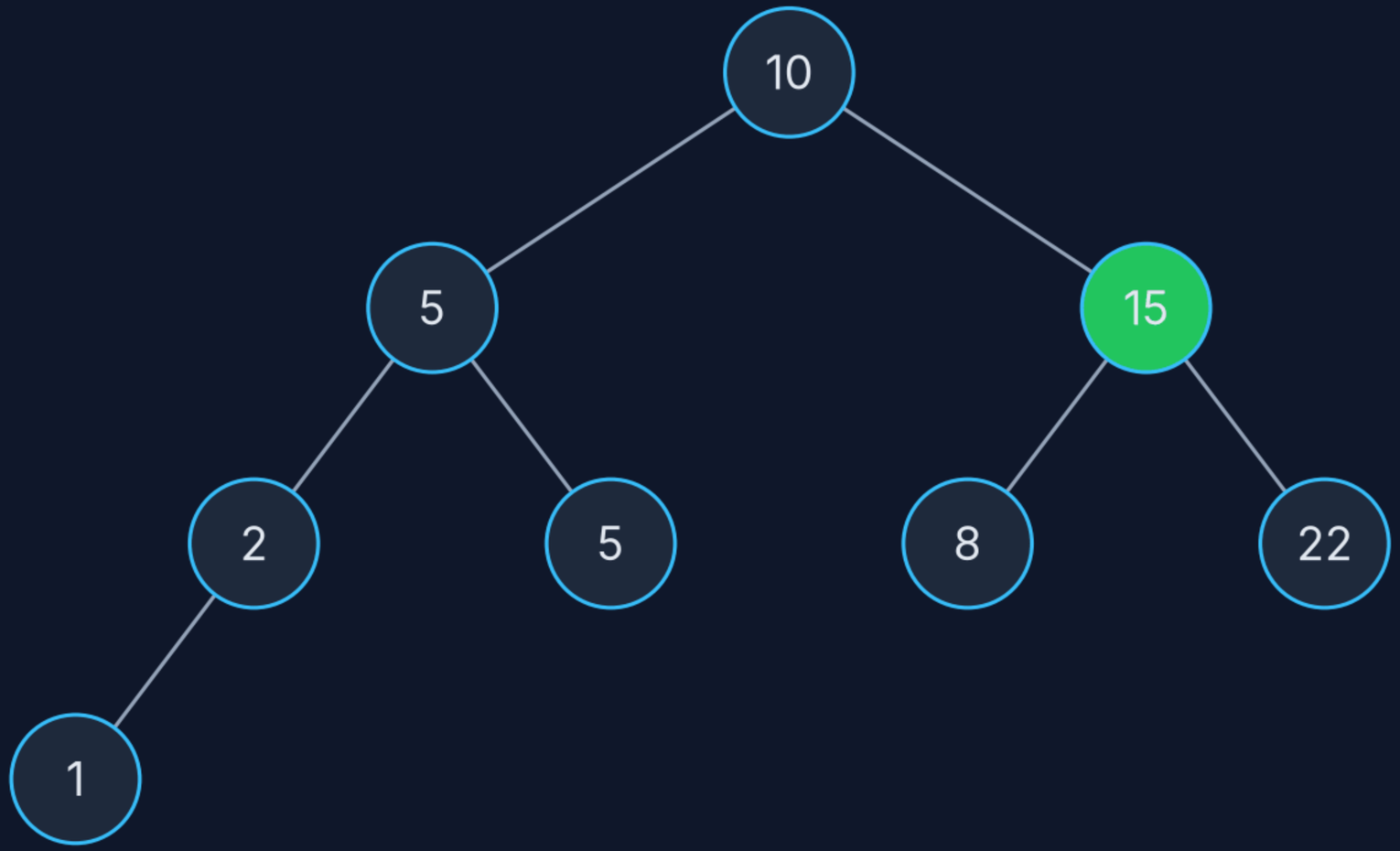



node 5

bounds [-∞, 10)

5 must be in [-∞, 10) → ok

# Worked example

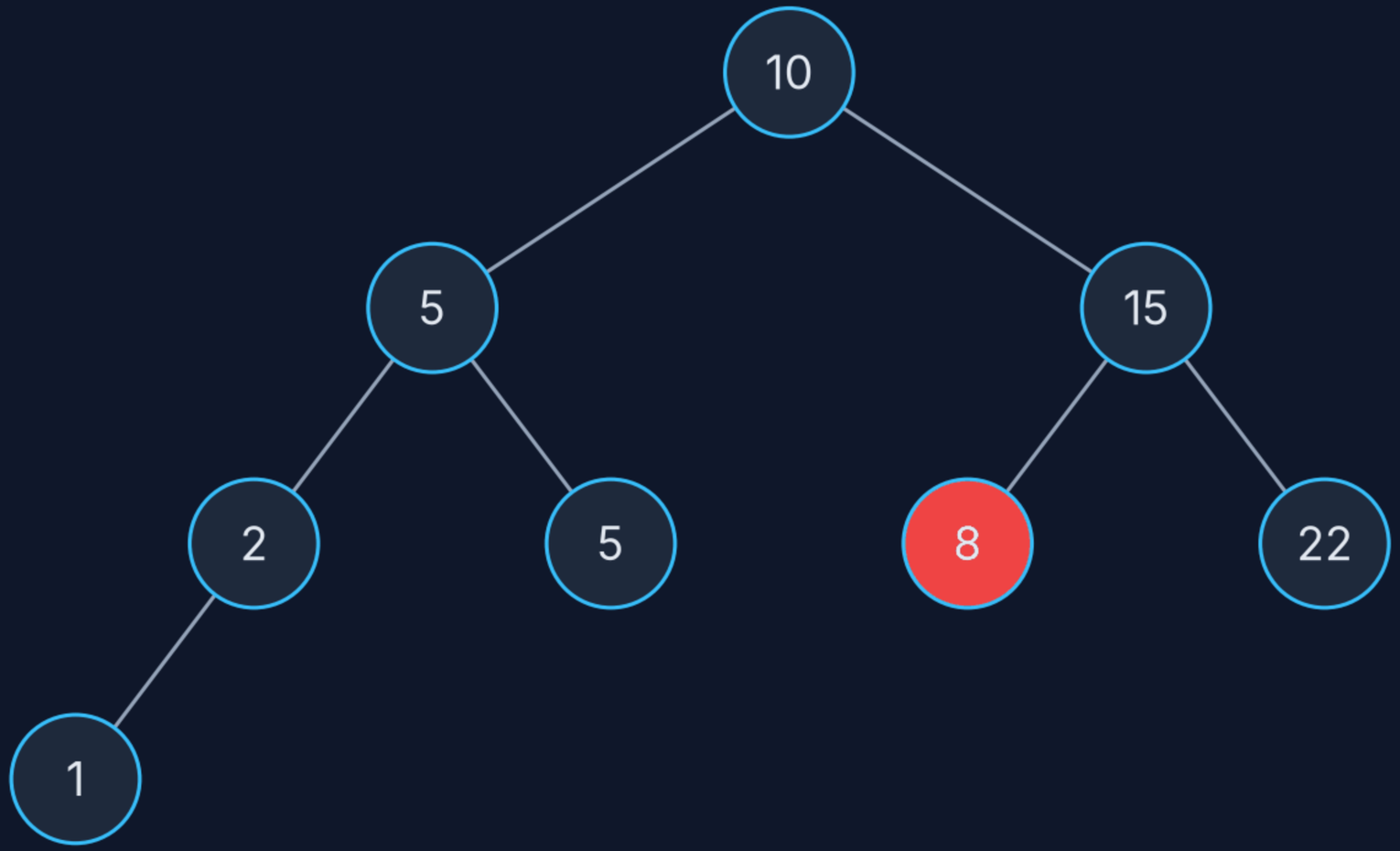



node 15

bounds [10, ∞)

15 must be in [10, ∞) → ok

# Worked example




node 8

bounds [10, 15)

8 must be in [10, 15) → INVALID

# Java solution

```java
public class ValidateBst {
    public static boolean validateBst(BST t) { return valid(t, Long.MIN_VALUE, Long.MAX_VALUE); }

    private static boolean valid(BST t, long min, long max) {
        if (t == null) return true;
        if (t.value < min || t.value >= max) return false;
        return valid(t.left, min, t.value) && valid(t.right, t.value, max);
    }
}
```

# Complexity

- Time: O(n)
- Space: O(h)

# Edge cases & pitfalls

- Grandchild violations (the classic bug)
- Duplicates go right: left bound inclusive
- Use long bounds for Integer.MIN/MAX values

# Interview tips & follow-ups

- Draw the counterexample before coding
- Follow-up: fix a BST with two swapped nodes (Repair BST)

# Recap

- Every node must lie within bounds inherited from its ancestors, not just compared with its parent.
- Complexity: O(n)